

Spring Security Introduction

1. Security Foundations

What Application Security Means

Suppose we have an endpoint:

```
DELETE /api/users/57
```

From a normal Spring MVC perspective, we think:

```
Request
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

From a security perspective, several questions must be answered **before the business operation is allowed**:

Who is sending **this** request?

Has that identity been verified?

Is that identity allowed **to** delete users?

Is that identity allowed **to** delete **this** particular user?

Has the request been manipulated?

Was the request genuinely created by the user?

Could an attacker repeat or forge this request?

Application security means protecting the application's valuable resources and operations from unauthorized access, modification, destruction, disclosure, or misuse.

It is not limited to protecting data.

An application may need to protect:

- information;
- business operations;
- user identities;
- system availability;
- financial transactions;
- administrative capabilities;
- secrets and credentials;
- application reputation.

For example, consider a banking application.

```
Asset: Account balance
Asset: Transaction history
Asset: Transfer-money operation
Asset: Customer identity
Asset: Bank availability
Asset: Administrative access
```

An attacker may not need to steal information. Making the bank unavailable through excessive requests is also a security problem.

Security is not one feature

Security is a collection of controls.

Authentication
 Authorization
 Input validation
 Session protection
 Password protection
 CSRF protection
 CORS policies
 Secure communication
 Rate limiting
 Audit logging
 Error handling
 Secret management

Spring Security primarily helps with:

Authentication
 Authorization
 Security-context management
 Session security
 Protection against several common web attacks
 Integration with security protocols

It does not magically make all application code secure. For example, Spring Security will not automatically fix

- SQL injection caused by unsafe queries;
- insecure file uploads;
- weak business rules;
- secrets committed to Git;
- vulnerable dependencies;
- incorrect database permissions;
- sensitive information written into logs.

Therefore:

Security is not something added only to the login page. It must exist wherever an untrusted actor can affect a valuable resource.

Assets, Threats, and Trust Boundaries

Before deciding how to secure an application, we need to understand three concepts:

- Assets
- Threats
- Trust boundaries

Assets

An asset is anything valuable that should be protected.

Data assets

- Passwords
- Email addresses
- Phone numbers
- Health records
- Payment information
- Source code
- API keys
- Business documents

Operational assets

- Creating an order
- Transferring money
- Approving a refund
- Changing another user's password

Deleting an account
Publishing content

Infrastructure assets

Database
Server
Cloud account
Message broker
File storage
Deployment pipeline

Security assets

Session IDs
JWT signing keys
Encryption keys
OAuth client secrets
Password-reset tokens
CSRF tokens

Availability as an asset

Even when an attacker cannot access data, they might attempt to prevent legitimate users from accessing the application.

Thousands of expensive requests
Repeated login attempts
Resource exhaustion
Database connection exhaustion

Therefore, availability itself is an asset.

Threats

A threat is something that could harm an asset.

Examples:

```
An attacker stealing credentials
A malicious employee reading restricted data
A bot repeatedly guessing passwords
A user changing another user's order
A script flooding an endpoint
A browser sending an unwanted authenticated request
```

A threat is not the same thing as a vulnerability.

Vulnerability

A vulnerability is a weakness that makes an attack possible.

```
Threat:
An attacker wants to access an administrator account.

Vulnerability:
The application allows unlimited password attempts.

Attack:
The attacker runs an automated password-guessing tool.

Impact:
The administrator account is compromised.
```

A useful relationship is:

```
Threat + Vulnerability → Possible Attack
```

Trust Boundaries

A trust boundary is a point where information moves from one trust level to another.

Consider:

```
Browser
  ↓
Internet
  ↓
Spring Boot application
  ↓
Database
```

Every transition represents a possible trust boundary.

```
Browser → Server
Server → Database
Service A → Service B
Mobile application → API gateway
Public network → Internal network
External identity provider → Application
```

Why is the browser untrusted?

A common beginner assumption is:

“The frontend only shows the delete button to admins, so normal users cannot delete anything.”

But users control their browsers.

They can:

- change JavaScript;
- modify HTML;
- use Postman;

- use `curl`;
- manipulate request parameters;
- manually call hidden endpoints;
- replay old requests.

Suppose the frontend contains:

```
if (user.role==="ADMIN") {
    showDeleteButton();
}
```

This improves the interface, but it does not provide security.

A non-admin user can still send:

```
DELETE /api/users/57
```

Therefore, authorization must be enforced on the server.

Trust-boundary rule

Whenever data crosses a trust boundary, ask:

```
Who produced this data?
Can it be modified?
Should it be authenticated?
Should it be authorized?
Should it be validated?
Should it be encrypted?
```

Authentication

Authentication answers:

Who are you, and what evidence proves that claim?

Suppose someone enters a company office and says:

| "I am Aditya."

That statement is only an **identity claim**.

The security guard may ask for an employee card.

```
Claimed identity: Aditya
Evidence: Employee card
Verification: Card is checked
Result: Identity accepted or rejected
```

That process is authentication.

Authentication is a process

Authentication is the process of verifying an identity claim.

Username and password example

```
Claim:
"My username is aditya."

Credential:
"My password is secret123."

Verification:
Compare the supplied password with the stored encoded password.

Result:
Authentication succeeds or fails.
```

The username usually identifies the account. The password provides evidence that the requester is entitled to use that account.

Authentication factors

Authentication evidence commonly falls into three broad categories.

Something you know

Password
PIN
Security answer

Something you have

Smart card
Authenticator application
Private cryptographic key

Something you are

Fingerprint
Face
Iris

Multi-Factor Authentication (MFA)

Using two credentials from the same category is not necessarily strong multi-factor authentication.

Password + Authenticator application OTP

uses knowledge and possession.

In Spring Security, this authenticated representation is usually stored in an `Authentication` object.

Authorization

Authorization answers:

| What is this identity allowed to do?

Authentication establishes identity.

Authorization evaluates access.

Suppose the security guard has verified that a person is Aditya. That does not mean Aditya can enter every room in the building.

Authentication:

You are **Aditya**.

Authorization:

Aditya may enter the training room,
but may not enter the finance department.

Authorization as a decision

A proper authorization decision usually needs four pieces of information:

Subject

Action

Resource

Context

Subject

Who is trying to perform the action?

User 51

Administrator

Manager

Service account

Anonymous visitor

Action

What are they trying to do?

Read
Create
Update
Delete
Approve
Transfer
Download

Resource

What are they trying to access?

Order 101
Employee record 57
Bank account 9001
Admin dashboard
Course video

Context

Under what conditions is the request happening?

Is the user the owner?
Is the request inside business hours?
Is the account active?
Is the amount below the approval limit?
Has MFA been completed?

Therefore, authorization can be thought of as:

Can subject **S**
perform action **A**

on resource **R**
under context **C?**

Example:

Can employee **27**
update
leave request **91**
when employee **27** owns leave request **91?**

Public access is also an authorization decision

Suppose an endpoint is public:

GET /api/courses/**public**

The application may allow everyone to access it.

That is still an authorization policy:

Authorization rule:
Permit all requests.

Authentication usually comes before authorization

A typical flow is:

```
Request
↓
Extract credentials
↓
Authenticate identity
↓
Determine authorities
↓
Evaluate authorization rule
↓
Allow or deny
```

Authentication Failures vs. Security Attacks

An authentication failure and a security attack are not necessarily the same thing.

Authentication failure

An authentication failure means the presented evidence could not establish an acceptable identity.

Examples

```
Wrong password
Unknown username
Expired token
Invalid token signature
Malformed Authorization header
Disabled account
Expired account
Expired session
Incorrect OTP
```

Security attack

An attack involves intentional attempts to defeat, misuse, bypass, or exploit security controls.

Brute-force attack

An attacker tries many passwords for one account.

```
aditya / password1
aditya / password2
aditya / password3
aditya / password4
```

Each individual request produces an authentication failure.

The pattern across thousands of requests reveals an attack.

Credential stuffing

An attacker uses username-password combinations stolen from another website.

```
Stolen credential:  
user@example.com / reusedPassword123
```

The password may even be correct.

Therefore, authentication could succeed even though the request is malicious.

This demonstrates an important point:

Successful authentication does not prove that the human behind the request is legitimate. It proves only that acceptable credentials were presented.

Additional controls may be needed:

```
MFA  
Anomaly detection  
Login notifications  
Device verification  
Rate limiting  
Credential-breach detection
```

Do not reveal unnecessary failure details

Suppose the application responds:

```
Username exists, but password is incorrect.
```

An attacker now knows that the account exists.

A safer external response is often:

```
Invalid username or password.
```

Internally, the application may log more precise information:

```
User not found  
Password mismatch  
Account disabled  
Token expired
```

The external message and internal diagnostic information need not be identical.

2. Spring Security Identity Model

Spring Security's identity model uses the following core vocabulary:

The important concepts are:

```
Identity  
Credentials  
Principal  
Authorities  
Roles  
Permissions  
Authentication  
SecurityContext  
SecurityContextHolder
```

They are related, but they are not interchangeable.

Identity

Identity means the entity whose actions the system wants to recognize.

It may represent:

```
A human user  
An administrator  
A customer  
Another application  
A microservice
```


A scheduled job
An IoT device

An identity is broader than a database row.

Identity versus account

An account is a stored application record.

users table

id	username	password_hash	enabled
42	aditya	...	true

Identity is the conceptual subject.

Account is one way the application represents and manages that identity.

A person may have:

- multiple accounts;
- an employee identity;
- a customer identity;
- an administrator identity;

Credentials

Credentials are evidence used to prove an identity claim.

Examples:

Password
OTP
Private key
Certificate
API key
Bearer Token

Principal

The principal is the identity currently represented inside the security system.

Authorities

An authority represents a capability or high-level security attribute granted to a principal.

Examples:

```
COURSE_READ  
COURSE_WRITE  
USER_DELETE  
PAYMENT_REFUND  
ROLE_ADMIN  
SCOPE_orders.read
```

In Spring Security, authorities are represented through `GrantedAuthority` objects and are stored inside the current `Authentication`. Spring describes authorities as high-level permissions such as roles and scopes.

Granted versus claimed authority

An authority should be trusted only when it has been assigned by a trusted authentication process.

A client should not be able to send:

```
{  
  "username": "aditya",  
  "role": "ADMIN"  
}
```

and become an administrator merely because it claimed that role.

The role or authority must be:

- loaded from a trusted database;
- derived from a verified token;
- supplied by a trusted identity provider;

- assigned by a trusted `AuthenticationProvider`.

The `Authentication` API explicitly treats its authorities as values granted by the `AuthenticationManager`, not arbitrary client claims.

Roles

A role is a named grouping of responsibilities or access privileges.

Examples:

```
USER
INSTRUCTOR
MANAGER
ADMIN
SUPER_ADMIN
```

A role is usually broader than an individual permission.

```
ROLE_INSTRUCTOR
├── COURSE_READ
├── COURSE_CREATE
├── COURSE_UPDATE
└── STUDENT_PROGRESS_READ
```

Authentication

`Authentication` is one of the most important Spring Security interfaces.

It has two different roles.

Role 1: Authentication request

Before credentials have been verified, an `Authentication` object can represent an authentication attempt.

Example:

```
UsernamePasswordAuthenticationToken
```

```
principal = "aditya"
credentials = "raw-password"
authorities = []
authenticated = false
```

This object means:

Someone claims to be Aditya and has supplied this password.

It does not yet mean the claim is accepted.

Role 2: Authenticated principal

After successful verification, another `Authentication` object can represent the accepted identity.

```
principal = UserDetails("aditya")
credentials = null
authorities = [ROLE_USER, COURSE_READ]
authenticated = true
```

This object means:

The authentication system has accepted this principal and granted these authorities.

Spring Security formally defines `Authentication` as both the token for an authentication request and the representation of an authenticated principal after processing.

Important `Authentication` information

Conceptually, it contains:

```
public interface Authentication {

    Object getPrincipal();

    Object getCredentials();

}
```

```
Collection<? extends GrantedAuthority> getAuthorities();

Object getDetails();

boolean isAuthenticated();

String getName();
}
```

principal

Represents the identity.

Before authentication:

```
"aditya"
```

After username-password authentication:

```
UserDetails object
```

credentials

The evidence proving the identity.

```
Password
Bearer token
Certificate
API key
```

Credentials may be removed after successful authentication.

authorities

The capabilities granted to the principal.

ROLE_USER
COURSE_READ
COURSE_WRITE

These are used later during authorization.

details

Additional information about the authentication request.

Depending on the mechanism, this may contain information such as:

Remote address
Session identifier

isAuthenticated()

This indicates whether the authentication token has been accepted as authenticated.

Conceptually:

Before verification:
isAuthenticated = false

After trusted verification:
isAuthenticated = true

Application code should not simply mark arbitrary client-created authentication objects as trusted. The authenticated result should normally come from the trusted authentication infrastructure.

Authentication transformation

Unauthenticated Authentication

principal	=	"aditya"
credentials	=	"raw-password"

```

| authorities = []
| authenticated = false

```

AuthenticationManager
verifies credentials



Authenticated Authentication

```

| principal = UserDetails
| credentials = null
| authorities = [ROLE_USER]
| authenticated = true

```

That authenticated object becomes the security identity used for the current request.

SecurityContext

The `SecurityContext` is a container that holds the current `Authentication`.

Conceptually:

```

public interface SecurityContext {

    Authentication getAuthentication();

    void setAuthentication(Authentication authentication);

}

```

Spring Security defines the `SecurityContext` as the object containing the current `Authentication`.

Why not store `Authentication` directly everywhere?

A wrapper provides a consistent place for request security state.

Instead of passing the current user manually:

```
courseService.createCourse(courseRequest, currentUser);
```

through every method, Spring Security establishes a request-associated security context that can be accessed where appropriate.

The security context can also be:

- loaded at the beginning of a request;
- updated after authentication;
- stored for future requests;
- cleared after request completion.

SecurityContextHolder

The `SecurityContextHolder` provides access to the current `SecurityContext`.

```
SecurityContext context =  
    SecurityContextHolder.getContext();  
  
Authentication authentication =  
    context.getAuthentication();
```

Then:

```
String username = authentication.getName();  
  
Object principal = authentication.getPrincipal();  
  
Collection<? extends GrantedAuthority> authorities =  
    authentication.getAuthorities();
```

Spring Security describes `SecurityContextHolder` as the central location where the framework stores details about the currently authenticated user.

Why is a holder necessary?

Consider this request:


```

HTTP request
  ↓
Security filters authenticate user
  ↓
Controller
  ↓
Service
  ↓
Method-security check
  
```

The authentication filter learns the user's identity.

Later components need access to that identity.

```

Authentication filter:
I authenticated Aditya.

Authorization component:
Who is the current user?

Controller:
Who submitted this request?

Audit service:
Which user performed this action?
  
```

The `SecurityContextHolder` gives these components access to the current security context without manually passing the user through every method.

Default `ThreadLocal` strategy

In servlet applications, one request is commonly processed by one thread at a time.

Spring Security therefore uses a `ThreadLocal` strategy by default.

Conceptually:

```
Thread handling request 1
  SecurityContext → Aditya
```

```
Thread handling request 2
  SecurityContext → Rohit
```

Each thread sees its own current context.

Spring Security documents that the default `SecurityContextHolder` strategy uses `ThreadLocal`, making the context available to methods running on the same thread. It also clears the context after request processing to prevent one request's identity from leaking into another request handled by a reused server thread.

Why clearing the context matters

Application servers reuse threads.

Without clearing:

```
Request 1
Thread 7
User = ADMIN
```

Then later:

```
Request 2
Same Thread 7
User = anonymous
```

If the old context remained, Request 2 might incorrectly inherit the administrator identity.

Therefore, the security context must be established and cleared around each request.

```
Request begins
↓
Load or create SecurityContext
```

```

↓
Process security and application logic
↓
Clear SecurityContextHolder
↓
Thread returns to pool

```

Complete identity-model relationship

```

Identity
|
| claimed using
▼
Credentials
|
| verified by authentication system
▼
Authentication
├── Principal
├── Credentials
├── Authorities
└── Authentication status
    |
    | stored inside
    ▼
SecurityContext
    |
    | accessed through
    ▼
SecurityContextHolder

```

3. Authentication State

HTTP creates an important problem.

By default, HTTP requests are independent.

Request 1

POST /login

Request 2

GET /profile

Request 3

GET /orders

Request 4

POST /logout

The server must decide:

How does Request 2 know that the user authenticated during Request 1?

There are two broad models:

Stateful authentication

Stateless authentication

Stateful Authentication

In stateful authentication, the server retains authentication-related state between requests.

A common example is session-based authentication.

User logs in once

↓

Server verifies credentials

↓

Server stores authentication in session

↓

Client receives session identifier

↓
Client sends session identifier on later requests
↓
Server retrieves stored authentication

First request: login

```
POST /login
Content-Type: application/x-www-form-urlencoded

username=aditya&password=secret
```

The application verifies the credentials.

Authentication successful

It then stores information representing the authenticated user in server-side session state.

Conceptually:

```
Session ID: ABC123

Session data:
  SecurityContext
    Authentication
      principal = aditya
      authorities = [ROLE_USER]
```

The response sends the session identifier to the browser, commonly using a cookie.

Set-Cookie: JSESSIONID=ABC123

Subsequent request

The browser sends:

```
GET /profile
Cookie: JSESSIONID=ABC123
```

The server uses the session identifier to retrieve the session.

```
ABC123
  ↓
Session
  ↓
SecurityContext
  ↓
Authentication for Aditya
```

The user does not need to send the password again.

Stateful does not mean all data is in the cookie

A common misunderstanding is:

“The browser cookie contains the complete logged-in user.”

In ordinary server-side session authentication, the cookie normally contains only a session identifier.

```
Browser:
JSESSIONID=ABC123

Server:
ABC123 → session containing SecurityContext
```

The important authentication state is held server-side.

The cookie allows the server to locate that state.

Where can session state be stored?

Session state may be kept in:

Application memory
Distributed cache
Redis
Database
External session store

Advantages of stateful authentication

User authenticates once

The user does not need to resend the original password on every request.

Immediate server-side invalidation

The server can remove or invalidate the session.

```
Delete session ABC123
```

Further requests using that session identifier fail.

Server controls session state

The application can record:

Last access time
Session expiry
Concurrent sessions
SecurityContext
Additional session attributes

Costs of stateful authentication

Server storage

Each active session consumes server-side resources.

Distributed-system complexity

When several servers handle requests, they need:

- shared session storage;
- sticky sessions;
- or another consistent session strategy.

Cookie does not necessarily mean session

Cookies may store many things:

```
Session identifier
Language preference
Theme preference
Analytics identifier
CSRF token
Authentication token
```

A token-based system can also place an access token inside a cookie.

Therefore:

Seeing a cookie does not automatically mean the application uses server-side session authentication.

Session cookie as a bearer credential

Suppose an attacker steals:

```
JSESSIONID=ABC123
```

The attacker may not know the user's password.

But the session identifier may be enough to retrieve the authenticated session.

In practice, possession of the session cookie can grant access, so it must be protected like a credential.

Important protections include:

```
HTTPS
Secure cookie attribute
HttpOnly
SameSite policy
Session expiration
Session invalidation
Session ID rotation
```

Session fixation

Imagine an attacker somehow causes a victim to use a session identifier known to the attacker.

```
Attacker knows session ID = XYZ
Victim logs in using session XYZ
Session XYZ becomes authenticated
Attacker reuses XYZ
```

To reduce this risk, the application can change the session identifier when authentication succeeds.

```
Before login:
Session = XYZ

After login:
Session = NEW-789
```

Spring Security includes session-fixation protection and can change the session ID after authentication.

Stateless Authentication

In stateless authentication, the server does not rely on a stored login session to identify the user on later requests.

Each request carries the information needed for authentication.

Request 1

Authorization: Bearer TOKEN

Request 2

Authorization: Bearer TOKEN

Request 3

Authorization: Bearer TOKEN

For every request:

Extract credential

↓

Validate credential

↓

Create Authentication

↓

Place it in SecurityContext

↓

Authorize request

↓

Clear request context

The next request starts again.

Stateless does not mean no data exists anywhere

A common incorrect statement is:

“A stateless system stores no state.”

The application may still store:

Users

Passwords

- Roles
- Permissions
- Signing keys
- Refresh tokens
- Revoked token records
- Audit logs
- Rate-limit counters
- Business data

Stateless authentication specifically means:

The server does not depend on a previously stored HTTP login session to reconstruct the user's authentication for the next request.

Tokens and Per-Request Authentication

A token is a credential presented by the client.

Example:

```
GET /api/profile
Authorization: Bearer eyJhbGciOi...
```

The server extracts the token and validates it.

- Check signature
- Check expiry
- Check issuer
- Check audience
- Read subject
- Determine authorities

It may then create:

```
Authentication
  principal = user 42
```

```
authorities = [SCOPE_profile.read]
authenticated = true
```

That authentication is used for only the current request unless the application explicitly persists it.

Self-Contained Token vs. Opaque Token

Self-contained token

A self-contained token contains signed information that the server can validate locally.

A JWT may contain claims such as:

```
{
  "sub": "aditya",
  "scope": "course.read course.write",
  "iat": 1785980000,
  "exp": 1785983600
}
```

The server validates the token using a trusted key.

It does not need to look up an HTTP session merely to identify the user.

Opaque token

An opaque token may look like:

```
a8f2b9137c45d...
```

The token itself does not reveal useful information to the application.

It is usually a random identifier.

The application may ask an authorization server or token store:

```
Is this token active?
Which user does it represent?
```

What scopes does it contain?

Opaque-token authentication can still be stateless from the application's HTTP-session perspective, even though a token service maintains token state.

Conceptually:

```
a8f2b9137c45d...
  ↓
Token database
  ↓
user = aditya
scope = course.read
active = true
expiry = 11:00
```

Bearer token

A bearer token effectively means:

| Whoever possesses this token may present it.

The word **Bearer** means that possession is enough.

The server generally does not require the client to prove that it is the original recipient of the token.

It is similar to cash:

Whoever possesses the cash can spend it.

The same observation applies to an ordinary session cookie. Someone possessing a valid **JSESSIONID** may be able to act as the user. In that practical sense, a session identifier behaves like a bearer credential.

Therefore, bearer tokens must be protected from:

- Logging
- URL leakage
- Browser storage attacks
- Network interception
- Accidental sharing
- Source-code commits

A valid token is not safe merely because it is a JWT.

Security still depends on:

- Strong signing keys
- Correct signature verification
- Expiry validation
- Issuer validation
- Audience validation
- Secure transport
- Safe client storage
- Appropriate scopes

An opaque token can be a bearer token

This is valid:

```
Authorization: Bearer a8f2b9137c45d...
```

Here:

- Bearer** = possession-based usage
- Opaque** = unreadable/reference format

The taxonomy

Access token

- ├─ Opaque
 - └─ Often used as a bearer token
- └─ Self-contained, often JWT
 - └─ Also often used as a bearer token

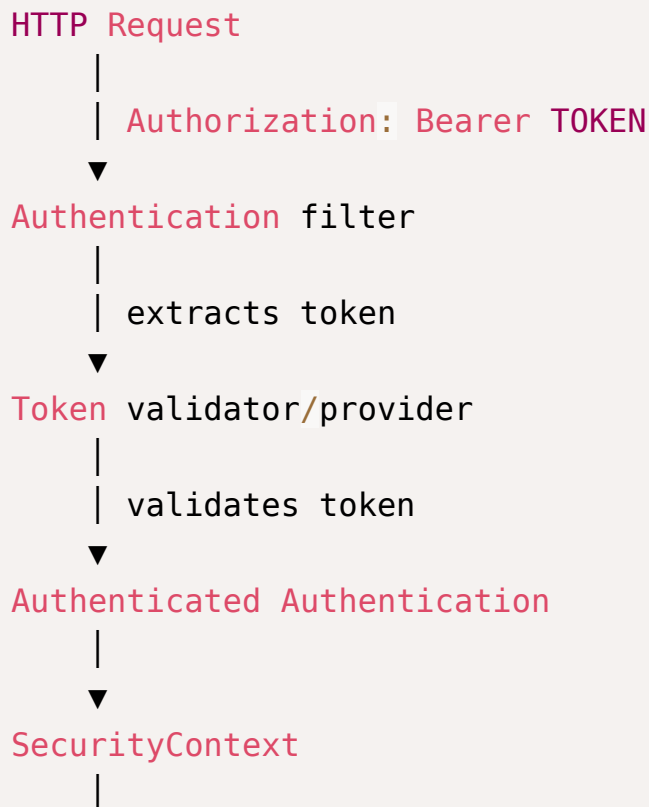
What Is a JWT?

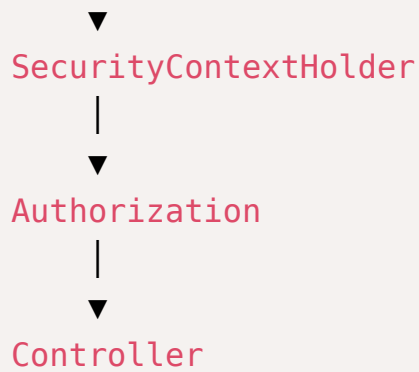
JWT stands for:

JSON Web Token

A JWT is a **token format**.

Per-request flow





At the end:

`SecurityContextHolder` cleared

On the next request, the token must be presented and processed again.

4. Authentication Mechanism vs. Session Policy

This is a fundamental distinction in Spring Security.

Authentication mechanism

The authentication mechanism answers:

| How are credentials obtained and verified?

Session policy

The session policy answers:

| After authentication succeeds, should the resulting `SecurityContext` be persisted in an HTTP session?

Form login describes how username and password are submitted.

`POST /login`

`username=aditya&password=secret`

The application may then:

Option A: Create a session

```
Form credentials
  ↓
Authenticate
  ↓
Store SecurityContext in session
  ↓
Send session cookie
```

This is the traditional stateful web application.

Option B: Issue a token

```
Form or JSON credentials
  ↓
Authenticate
  ↓
Generate access token
  ↓
Do not create login session
```

This can form part of a stateless API design.

Similarly, HTTP Basic is a credential transport mechanism

HTTP Basic sends credentials through the `Authorization` header.

```
Authorization: Basic YWRpdHlhOnNlY3JldA==
```

Usually the client presents the header repeatedly.

That naturally fits per-request authentication.

But the authentication mechanism and session policy are still conceptually separate. An application could authenticate Basic credentials and persist the resulting context unless configured not to do so.

Therefore:

HTTP Basic

= credential transport mechanism

STATELESS

= session persistence decision

JWT is not automatically stateless

A JWT is a token format.

Statelessness depends on how the application uses it.

Stateless use

```
JWT sent every request
JWT validated every request
No HTTP-session authentication used
```

Session-backed use

```
JWT validated once
Authentication stored in HTTP session
Later requests use session cookie
```

The second design uses a JWT during authentication but becomes session-based afterward.

Therefore:

| Token format does not determine session policy.

Conversely, a non-JWT opaque token can be processed on every request without creating an HTTP session.

Cookie does not automatically mean stateful

A stateless application may store a bearer token in a secure cookie.

```
Cookie: access_token=eyJhbGciOi...
```

On every request:

```
Read token from cookie
Validate token
Create Authentication
No server-side login session
```

This is still stateless with respect to authentication persistence, even though a cookie transports the token.

However, automatically submitted cookies introduce browser-specific security concerns such as CSRF, which must be addressed separately.

Stateful Request Lifecycle

Initial authentication

```
POST /login
↓
Username and password extracted
↓
Credentials authenticated
↓
Authentication created
↓
SecurityContext created
↓
SecurityContext saved in HttpSession
```

↓
Session cookie returned

Later request

```
GET /profile
Cookie: JSESSIONID=ABC123
↓
Session loaded
↓
SecurityContext loaded
↓
Authentication placed in SecurityContextHolder
↓
Authorization performed
↓
Controller executes
↓
SecurityContextHolder cleared
```

Stateless Request Lifecycle

```
GET /profile
Authorization: Bearer TOKEN
↓
Token extracted
↓
Token authenticated
↓
Authentication created
↓
SecurityContext created
↓
Authentication placed in SecurityContextHolder
↓
```

Authorization performed

↓

Controller executes

↓

SecurityContextHolder cleared

↓

No authentication saved in HTTP session

Next request:

Start the authentication process again
using the credential carried by that request.

5. Spring Boot Security Auto-Configuration

Spring Boot's job is not to implement authentication itself.

Spring Security provides the security framework.

Spring Boot's job is to observe what is available and create a sensible initial configuration.

A simplified mental model is:

Dependency added

↓

Spring Security classes found on classpath

↓

Spring Boot security auto-configuration activates

↓

Security beans are created

↓

Security filter chain is registered

↓

Every matching request passes through security

Spring Boot's servlet security defaults are primarily coordinated through:

`SecurityAutoConfiguration`

`UserDetailsServiceAutoConfiguration`

`SecurityAutoConfiguration` provides the default web-security setup, while `UserDetailsServiceAutoConfiguration` provides the default username-and-password user configuration.

Spring Boot is solving two different problems:

Problem 1:

Which requests should be `protected`,
and which authentication mechanisms should be available ?

Problem 2:

Which user can authenticate,
and where will that user be loaded from?

Those problems correspond conceptually to:

`Default SecurityFilterChain`

+

`Default UserDetailsService`

The default security configuration

The effective default security configuration can be understood approximately as:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.authorizeHttpRequests(
        authorize -> authorize
```

```
        .anyRequest()  
        .authenticated()  
    )  
    .formLogin(Customizer.withDefaults())  
    .httpBasic(Customizer.withDefaults());  
  
    return http.build();  
}
```

This means:

```
anyRequest().authenticated()
```

Every request requires authentication.

```
formLogin()
```

Username-password authentication through an HTML login form is available.

```
httpBasic()
```

Username-password authentication through the HTTP `Authorization` header is also available.

Spring Security documents this as the default filter-chain configuration: every request requires authentication, while both form login and HTTP Basic are enabled.

Auto-configuration is conditional

Spring Boot auto-configuration follows an important rule:

Provide defaults only when the developer has not provided their own configuration.

Suppose Spring Boot asks:

Has the developer created a `SecurityFilterChain` bean?

If the answer is no:

Create the `default` web-security configuration.

If the answer is yes:

Back off and let the developer control web security.

Therefore, when we later write:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    // Our configuration

}
```

Spring Boot stops supplying its default web-security rules.

But there is an important nuance:

Creating a `SecurityFilterChain` bean replaces the default web-security configuration, but it does not automatically remove the generated default user.

To replace the generated user configuration as well, we normally provide one of the following:

```
UserDetailsService
AuthenticationProvider
AuthenticationManager
```

Spring Boot documents these as separate back-off conditions: a custom `SecurityFilterChain` replaces the default access configuration, while a custom user

or authentication component replaces the generated user setup.

So these are independent:

`SecurityFilterChain`

→ **How** requests are secured

`UserDetailsService` / `AuthenticationProvider`

→ **How** users are authenticated

6. Default User

Spring Boot needs at least one identity that can authenticate.

Without a user, the application would present a login page, but no credentials could ever succeed.

Therefore, Spring Boot creates a default in-memory user.

The default username is:

```
user
```

The user is stored in memory through a `UserDetailsService`, rather than being inserted into a database. Spring Boot's default user store contains one user, and the default username is `user`.

Conceptually:

`InMemoryUserDetailsManager`

└─ `UserDetails`

└─ `username` = `"user"`

└─ `password` = generated password

└─ `authorities` = `["ROLE_USER"]`

└─ `enabled` = `true`

This user is not:

- saved in MySQL;
- saved through JPA;
- created by your controller;
- a permanent production account;
- related to a registration system.

It exists only because Spring Boot needs a development-time identity for testing its default security configuration.

What happens when we enter `user` ?

The login request provides:

```
Principal claim:
user

Credential:
password entered in the login form
```

Spring Security then asks the configured user store:

```
Load the user whose username is "user".
```

The in-memory user store returns something conceptually like:

```
UserDetails
  username = user
  password = encoded stored password
  authorities = ROLE_USER
```

The submitted password is checked against the stored password representation.

If it matches:

```
Authentication successful
```

Spring Security creates an authenticated `Authentication` object:

```
Authentication
├─ principal = UserDetails("user")
├─ credentials = cleared
├─ authorities = [ROLE_USER]
└─ authenticated = true
```

That authentication is then placed in the `SecurityContext`.

Default role

The default user normally has:

```
ROLE_USER
```

Remember the distinction:

```
Role name:
USER
```

```
Granted authority:
ROLE_USER
```

For the default rule:

```
anyRequest().authenticated()
```

the actual role does not matter.

Spring Security is only asking:

```
Is there an accepted authenticated identity?
```

It is not yet asking:

Does the user have `ROLE_ADMIN`?

Therefore, the default user can access every endpoint after authentication because the default rule requires authentication, not a particular authority.

Configuring the default user through properties

For demonstration purposes, we can replace the generated credentials:

```
spring.security.user.name=aditya
spring.security.user.password=secret123
spring.security.user.roles=USER, ADMIN
```

Now the user becomes conceptually:

```
username = aditya
password = secret123
authorities = ROLE_USER, ROLE_ADMIN
```

7. Generated Password

When the application starts, the console displays something similar to:

```
Using generated security password:
78fa095d-3f4c-48b1-ad50-e24c31d5cf35
```

This generated password is for development use only.
Your security configuration must be updated before running your application in production.

The generated value is the password that must be entered with:

```
Username: user
Password: generated value
```

Spring Boot generates a random default password and logs it at `WARN` level during startup. The documentation explicitly describes this password as development-only.

8. The CSRF `403 Forbidden` Surprise

Suppose a user successfully logs in and then sends:

```
POST /api/courses
```

They may receive:

```
403 Forbidden
```

They might conclude:

“The user does not have permission.”

But that is not necessarily the reason.

Spring Security enables CSRF protection by default for servlet web applications. Therefore, a state-changing request such as `POST`, `PUT`, `PATCH`, or `DELETE` may be rejected when it lacks the expected CSRF token.

The request may have:

```
Valid authenticated user  
Valid ROLE_USER  
Correct endpoint
```

but still fail because:

```
Required CSRF proof is missing
```

Key takeaway: `401 Unauthorized` usually means acceptable authentication is missing or has failed. `403 Forbidden` means the request was understood but

rejected. With Spring Security, a 403 on a state-changing request may be caused by a missing CSRF token—not only by insufficient roles or authorities.
